UNDERGRADUATE COMPUTER NETWORKS LAB

Dynamic Routing

RIP & OSPF

Configure, observe, recover

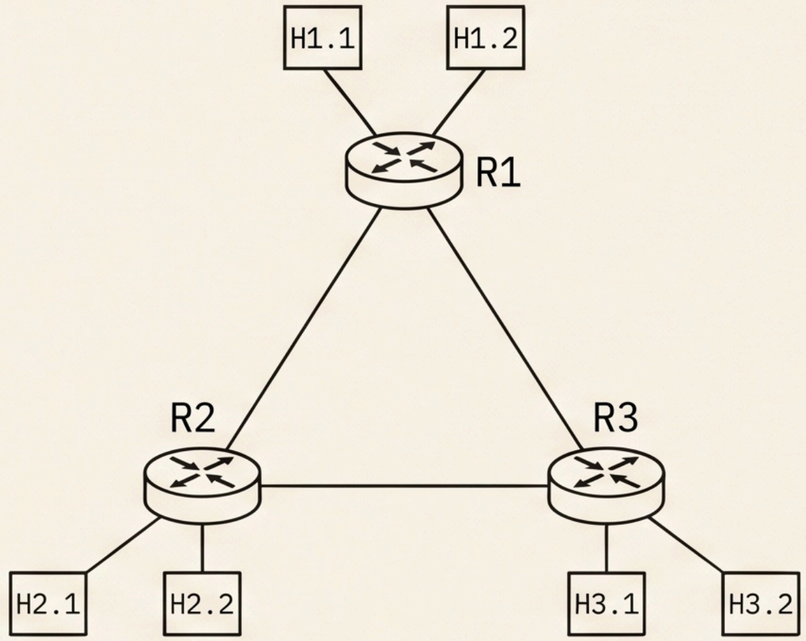
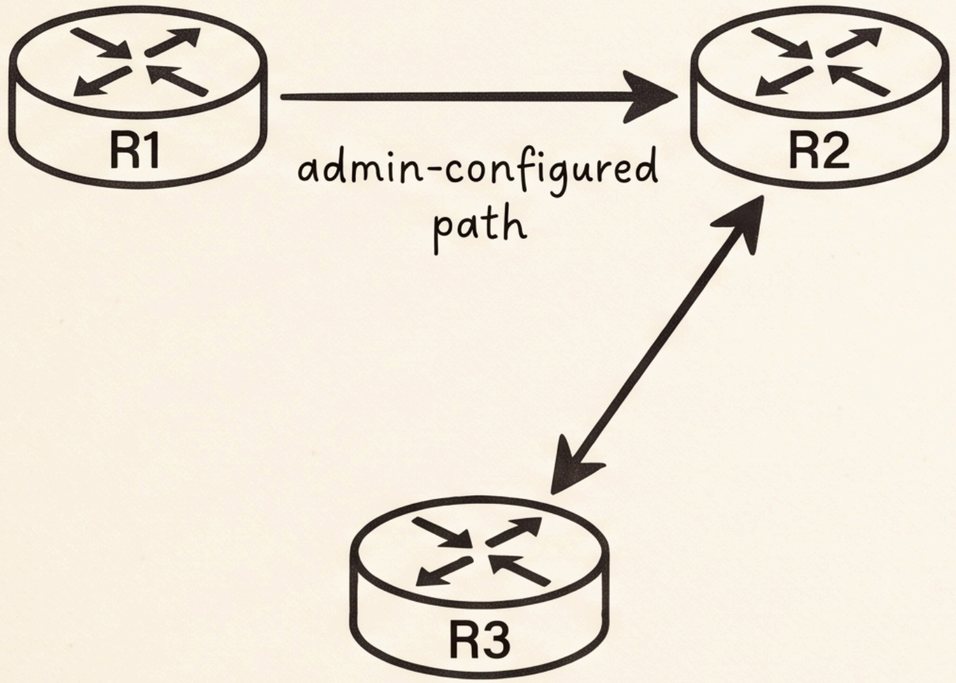


FIG. 1: THREE-NODE REDUNDANT TOPOLOGY FOR RIP/OSPF TESTING

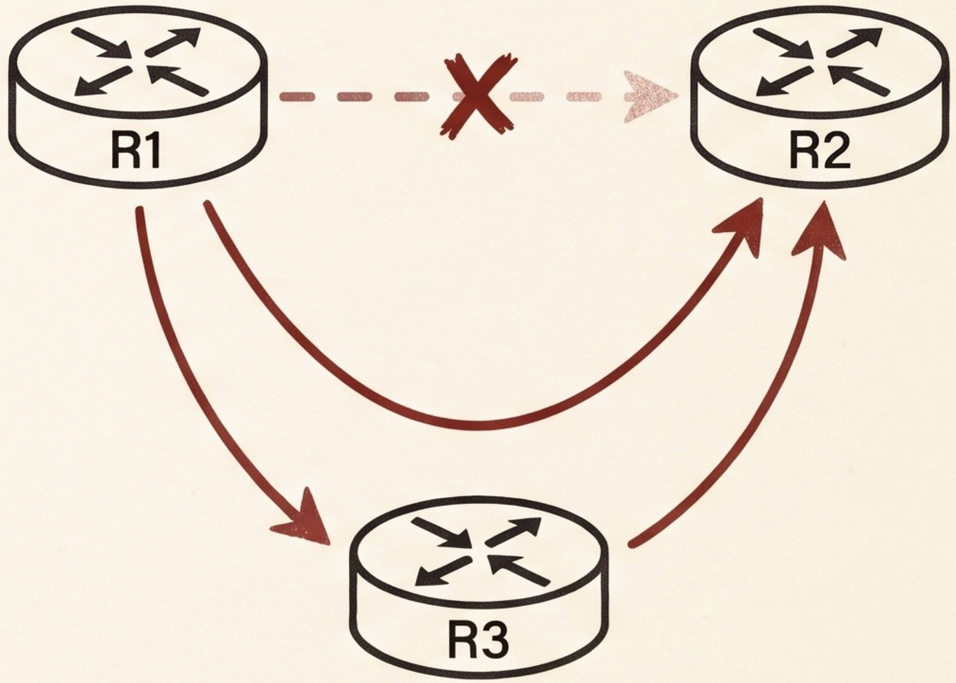
‘Why dynamic routing?

STATIC



Static — fixed admin paths

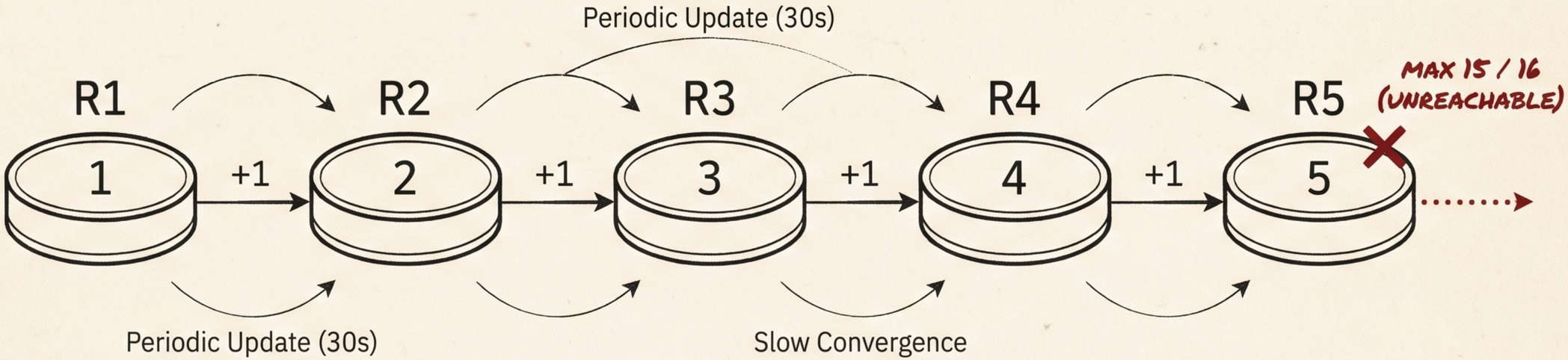
DYNAMIC



Dynamic — adapts to change

RIP — Distance Vector

Hop-count metric



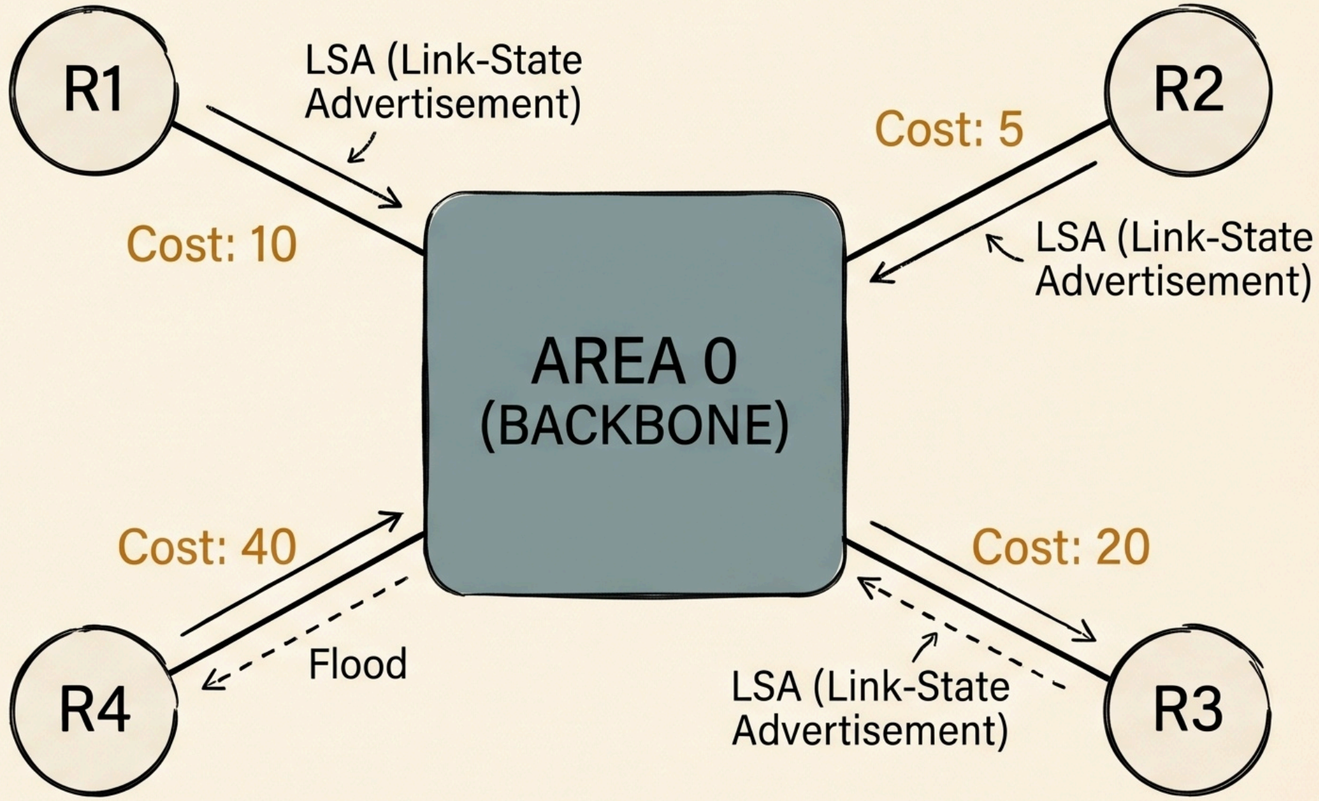
Statistics

Update 30s · Invalid 180s · RFC 1058 / RFC 2453

OSPF — Link State

Cost-based, hierarchical areas

- *Uses Link-State Advertisements (LSAs) to build a topology map.*
- *Computes the shortest paths using Dijkstra's SPF Algorithm.*
- *Organizes the network hierarchically around a central Backbone (Area 0).*



RFC 2328 · Cost = ref-bw / bw · Hello 10s / Dead 40s

RIP *vs* OSPF

RIP

ROUTING INFORMATION PROTOCOL

- Distance-vector
- Hop count, 1 to 15 `(MAX 15)`
- Slow convergence
- Best for small networks

RFC 1058 · RFC 2453

OSPF

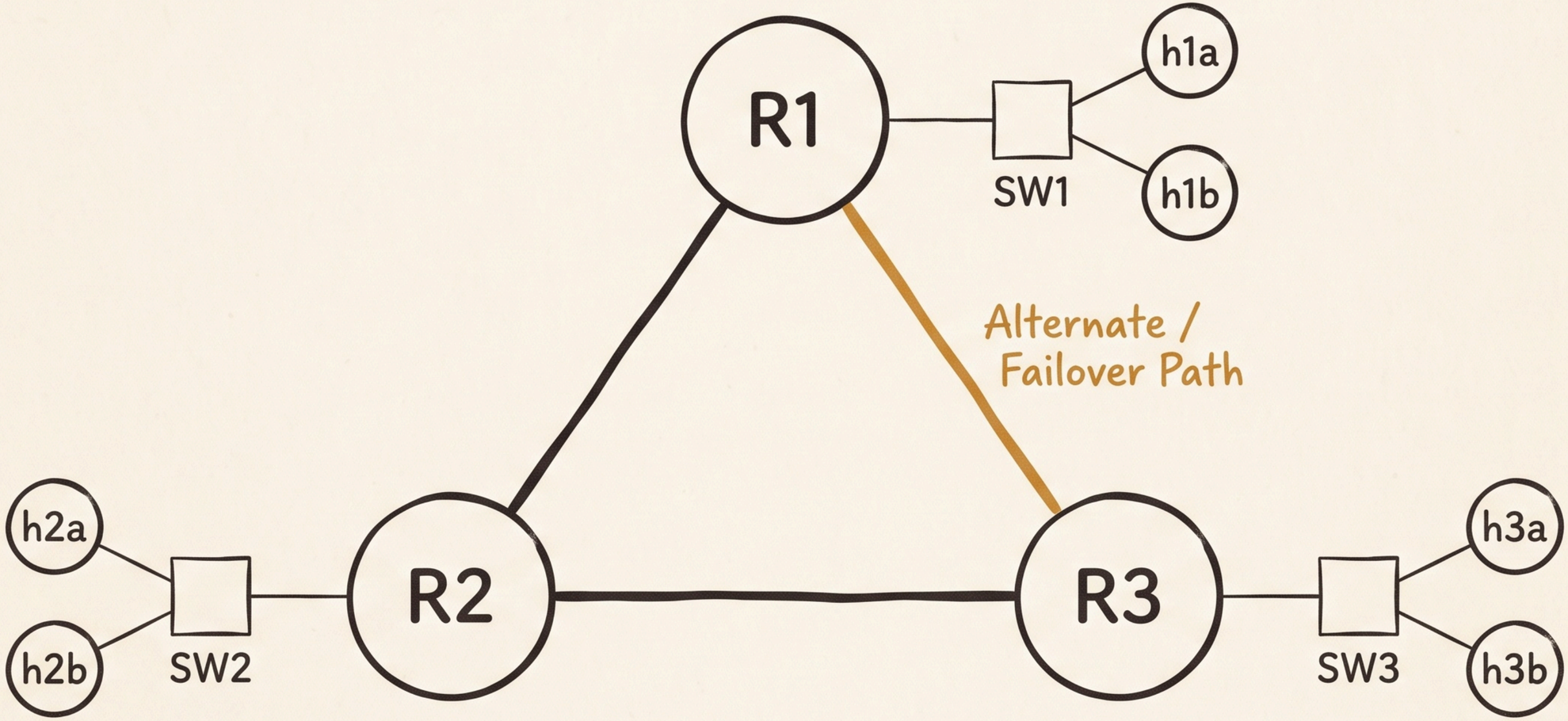
OPEN SHORTEST PATH FIRST

- Link-state
- `Cost = ref-bw / bw`
- Faster convergence
- Hierarchical areas

RFC 2328

Triangle of Three Routers

Each router is a Linux namespace running FRR



Mininet — Custom Router Topology

three Linux-router namespaces

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel

class TriangleRouters(Topo):
    def build(self):
        # 1. Add Routers (Linux namespaces)
        r1 = self.addHost('r1', ip='10.0.1.1/24')
        r2 = self.addHost('r2', ip='10.0.2.1/24')
        r3 = self.addHost('r3', ip='10.0.3.1/24')

        # 2. Add Switches & End Hosts
        s1, s2, s3 = self.addSwitch('s1'),
        self.addSwitch('s2'),
        self.addSwitch('s3')
        h1a = self.addHost('h1a', ip='10.1.1.2/24',
              defaultRoute='via 10.1.1.1')
        h1b = self.addHost('h1a', ip='10.1.1.2/24',
        h1b = self.addHost('h1b', ip='10.1.1.3/24',
        h2a = self.addHost('h2a', ip='10.2.2.2/24',
        h2a = self.addHost('h2a', ip='10.2.2.1')
        h2b = self.addHost('h2b', ip='10.2.2.3/24',
        h2b = self.addHost('h2b', ip='10.2.2.1')
        h3a = self.addHost('h3a', ip='10.3.3.2/24',
        h3b = self.addHost('h3a', ip='10.3.3.1')
        h3b = self.addHost('h3b', ip='10.3.3.3/24',
              defaultRoute='via 10.3.3.1')
        h2a = self.addHost('h2a', ip= defaultRoute='via 10.2.2.1')
        h2b = self.addHost('h2b', ip= defaultRoute='via 10.2.2.1')
        h3a = self.addHost('h3a', ip= defaultRoute='via 10.3.3.1')
        h3b = self.addHost('h3b', ip= defaultRoute='via 10.3.3.1')

        # 3. Wire LANs
        self.addLink(h1a, s1)
        self.addLink(h1b, s1)
        self.addLink(s1, r1)
        self.addLink(h2a, s2)
        self.addLink(h2b, s2)
        self.addLink(s2, r2)
        self.addLink(h3a, s3)
        self.addLink(h3b, s3)
        self.addLink(h3b, s3)
        self.addLink(s3, r3)

        # 4. Wire Triangle Backhaul
        self.addLink(r1, r2)
        self.addLink(r2, r3)
        self.addLink(r1, r3)  # Alternate R1-R3 link

if __name__ == '__main__':
    setLogLevel('info')
    topo = TriangleRouters()
    net = Mininet(topo=topo)
    net.start()
    CLI(net)
    net.stop()
```

Standard margin headers

paginatin margins

FRR — RIP Configuration

router rip · version 2

enable in /etc/frr/daemons	zebra=yes · ripd=yes

```
router rip
   version 2
   network 10.0.0.0/8
   network 192.0.2.0/24
```

Apply · sudo systemctl restart frr — enter vtysh

FRR — OSPF Configuration

router ospf · area 0

enable in /etc/frr/daemons — zebra=yes · ospfd=yes

```
router ospf
    ospf router-id 1.1.1.1
    network 10.0.0.0/8 area 0.0.0.0
    network 192.0.2.0/24 area 0.0.0.0
```

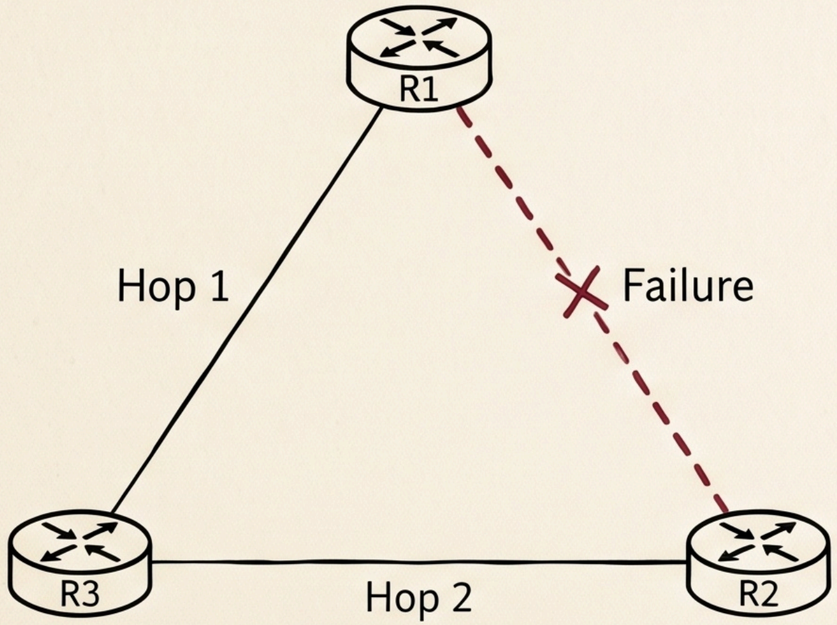
Apply · sudo systemctl restart frr — vtysh — show ip ospf neighbor

CONVERGENCE & RECOVERY

Route Recovery After Link Failure (R1–R2) in a Triangle Topology

RIP

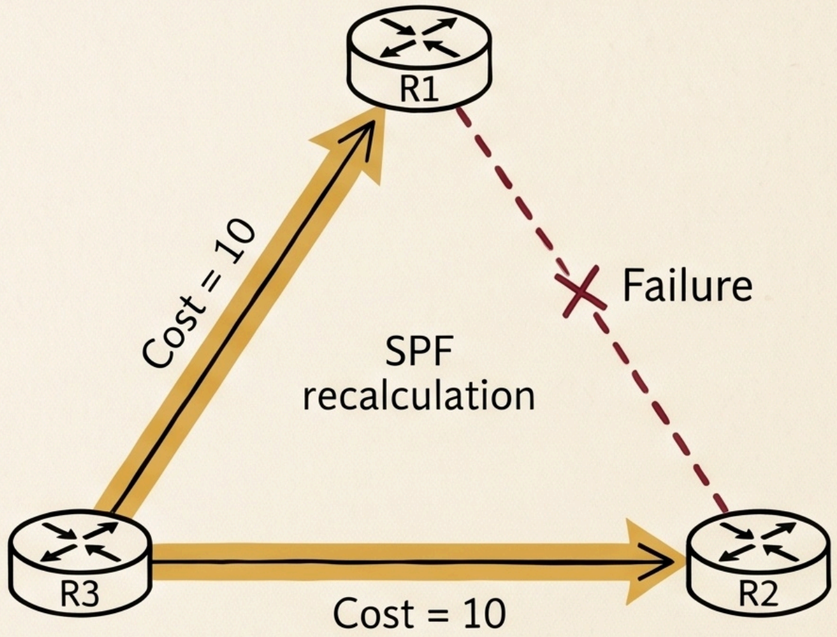
Slow graceful degradation



Hops increment slowly
(1 → 2 → 3…)

OSPF

Faster SPF recomputation



Rapid convergence
via Link-State packets